# P1040R4
# std::embed

Published Proposal, 2019-01-21

**Author:**
JeanHeyd Meneide

**Audience:**
EWG, LEWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Latest:**
https://thephd.github.io/vendor/future_cxx/papers/d1040.html

**Implementation:**
https://github.com/ThePhD/embed-implementation

**Reply To:**
JeanHeyd Meneide, @thephantomderp

---

## Abstract

A proposal for a function that allows pulling resources at compile-time into a program.

## Table of Contents

**1 Revision History**
1.1 Revision 4 - November 26th, 2018
1.2 Revision 3 - November 26th, 2018
1.3 Revision 2 - October 10th, 2018
1.4 Revision 1 - June 10th, 2018
1.5 Revision 0 - May 11th, 2018

**2 Motivation**

**3 Scope and Impact**

**4 Design Decisions**
4.1 Current Practice
4.1.1 Manual Work
4.1.2 Processing Tools
4.1.3 `ld`, resource files, and other vendor-specific link-time tools
4.1.4 The `incbin` tool
4.2 Prior Art
4.2.1 Literal-Based, constexpr
4.2.2 Literal-Based, Null Terminated (?)

4.2.3 Encoding
4.3 Design Goals
4.3.1 Implementation Defined
4.3.2 Binary Only
4.3.3 Constexpr Compatibility
4.3.4 Statically Polymorphic

**5 Changes to the Standard**
5.1 Intent
5.2 Proposed Feature Test Macro
5.3 Proposed Wording

**6 Appendix**
6.1 Sadness
6.1.1 Pre-Processing Tools Sadness
6.1.2 `python` Sadness
6.1.3 `ld` Sadness

**7 Acknowledgements**

**References**
Informative References

# § 1. Revision History

## § 1.1. Revision 4 - November 26th, 2018

— Wording is now relative to [n4778].

— Minor typo and tweak fixes.

## § 1.2. Revision 3 - November 26th, 2018

— Change to using `consteval`.

— Discuss potential issues with accessing resources after full semantic analysis is performed. Prepare to poll Evolution Working Group. Reference new paper, [p1130], about resource management.

## § 1.3. Revision 2 - October 10th, 2018

— Destroy `embed_options` and `alignment` options: if the function is materialized only at compile-time through `constexpr` or the upcoming "immediate functions" (`constexpr!`), there is no reason to make this part of the function. Instead, the user can choose their own alignment when they pin this down into a std::array or some form of C array / C++ storage.

## § 1.4. Revision 1 - June 10th, 2018

— Create future directions section, follow up on Library Evolution Working Group comments.

— Change `std::embed_options::null_terminated` to `std::embed_options::null_terminate`.

— Add more code demonstrating the old way and motivating examples.

— Incorporate LEWG feedback, particularly alignment requirements illuminated by Odin Holmes and Niall Douglass. Add a feature macro on top of having `__has_include( <embed> )`.

### § 1.5. Revision 0 - May 11th, 2018

Initial release.

## § 2. Motivation

> I'm very keen on std::embed. I've been hand-embedding data in executables for NEARLY FORTY YEARS now. — *Guy "Hatcat" Davidson, June 15, 2018*

**Currently**

```
se-shell@virt-deb> python strfy.py \
        fxaa.spriv \
        stringified_fxaa.spirv.h
```

**With Proposal**

```
#include <embed>

int main (int, char*[]) {
        constexpr std::span<const std::byte
        fxaa_binary =
                std::embed( "fxaa.spirv" )

        // assert this is a SPIRV
        // file, at compile-time
        static_assert(fxaa_binary[0] == 0x0
                && fxaa_binary[1] == 0x02
                && fxaa_binary[2] == 0x23
                && fxaa_binary[3] == 0x07,
                "given wrong SPIRV data, "
                "check rebuild or check "
                "the binaries!")

        auto context = make_vulkan_context

        // data kept around and made
        // available for binary
        // to use at runtime
        auto fxaa_shader = make_shader(
                context, fxaa_binary );

        for (;;) {
                // ...
                // and we're off!
                // ...
        }

        return 0;
}
```

```
#include <span>

constexpr inline
const auto& fxaa_spriv_data =
#include "stringified_fxaa.spirv.h"
;

// prevent embedded nulls from
// ruining everything with
// char_traits<char>::length
// or strlen
template <typename T, std::size_t N>
constexpr std::size_t
string_array_size(const T (&)[N]) {
    return N - 1;
}

int main (int char*[]) {
        constexpr std::span<const std::byte>
        fxaa_binary{
                fxaa_spriv_data,
                string_array_size(fxaa_spriv_data)
        };

        // assert this is a SPIRV
        // file, at compile-time
        static_assert(fxaa_binary[0] == 0x03
                && fxaa_binary[1] == 0x02
                && fxaa_binary[2] == 0x23
                && fxaa_binary[3] == 0x07,
                "given wrong SPIRV data, "
                "check rebuild or check "
                "the binaries!")

        auto context = make_vulkan_context();

        // data kept around and made
        // available for binary
        // to use at runtime
        auto fxaa_shader = make_shader(
                context, fxaa_binary );

        for (;;) {
                // ...
                // and we're off!
                // ...
        }

        return 0;
}
```

Every C and C++ programmer -- at some point -- attempts to `#include` large chunks of non-C++ data into their code. Of course, `#include` expects the format of the data to be source code, and thusly the program fails with spectacular lexer errors. Thusly, many different tools and practices were adapted to handle this, as far back as 1995 with the `xxd` tool. Many industries need such functionality, including (but hardly limited to):

— Financial Development

  — representing coefficients and numeric constants for performance-critical algorithms;

— Game Development

  — assets that do not change at runtime, such as icons, fixed textures and other data

  — Shader and scripting code;

— Embedded Development

  — storing large chunks of binary, such as firmware, in a well-compressed format

  — placing data in memory on chips and systems that do not have an operating system or file system;

— Application Development

  — compressed binary blobs representing data

  — non-C++ script code that is not changed at runtime; and

— Server Development

  — configuration parameters which are known at build-time and are baked in to set limits and give compile-time information to tweak performance under certain loads

  — SSL/TLS Certificates hard-coded into your executable (requiring a rebuild and potential authorization before deploying new certificates).

In the pursuit of this goal, these tools have proven to have inadequacies and contribute poorly to the C++ development cycle as it continues to scale up for larger and better low-end devices and high-performance machines, bogging developers down with menial build tasks and trying to cover-up disappointing differences between platforms.

MongoDB has been kind enough to share some of their code below. Other companies have had their example code anonymized or simply not included directly out of shame for the things they need to do to support their workflows. The author thanks MongoDB for their courage and their support for `std::embed`.

The request for some form of `#include_string` or similar dates back quite a long time, with one of the oldest stack overflow questions asked-and-answered about it dating back nearly 10 years. Predating even that is a plethora of mailing list posts and forum posts asking how to get script code and other things that are not likely to change into the binary.

This paper proposes `<embed>` to make this process much more efficient, portable, and streamlined.

## § 3. Scope and Impact

`consteval span<const byte> embed( string_view resource_identifier )` is an extension to the language proposed entirely as a library construct. The goal is to have it implemented with compiler intrinsics, builtins, or other suitable mechanisms. It does not affect the language. The proposed header to expose this functionality is `<embed>`, making the feature entirely-opt-in by checking if either the proposed feature test macro or header exists.

## § 4. Design Decisions

`<embed>` avoids using the preprocessor or defining new string literal syntax like its predecessors, preferring the use of a free function in the `std` namespace. `<embed>`'s design is derived heavily from community feedback plus the rejection of the prior art up to this point, as well as the community needs demonstrated by existing practice and their pit falls.

## § 4.1. Current Practice

Here, we examine current practice, their benefits, and their pitfalls. There are a few cross-platform (and not-so-cross-platform) paths for getting data into an executable.

### § 4.1.1. Manual Work

Many developers also hand-wrap their files in (raw) string literals, or similar to massage their data -- binary or not -- into a conforming representation that can be parsed at source code:

0. Have a file `data.json` with some data, for example:

```
{ "Hello": "World!" }
```

1. Mangle that file with raw string literals, and save it as `raw_include_data.h`:

```
R"json({ "Hello": "World!" })json"
```

2. Include it into a variable, optionally made `constexpr`, and use it in the program:

```
#include <iostream>
#include <string_view>

int main() {
  constexpr std::string_view json_view =
#include "raw_include_data.h"
    ;

  // { "Hello": "World!" }
  std::cout << json_view << std::endl;
  return 0;
}
```

This happens often in the case of people who have not yet taken the "add a build step" mantra to heart. The biggest problem is that the above C++-ready source file is no longer valid in as its original representation, meaning the file as-is cannot be passed to any validation tools, schema checkers, or otherwise. This hurts the portability and interop story of C++ with other tools and languages.

Furthermore, if the string literal is too big vendors such as VC++ will hard error
the build (example from Nonius, benchmarking framework).

### § 4.1.2. Processing Tools

Other developers use pre-processors for data that can't be easily hacked into a C++ source-code appropriate state (e.g., binary). The most popular one is `xxd -i my_data.bin`, which outputs an array in a file which developers then include. This is problematic because it turns binary data in C++ source. In many cases, this results in a larger file due to having to restructure the data to fit grammar requirements. It also results in needing an extra build step, which throws any programmer immediately at the mercy of build tools and project management. An example and further analysis can be found in the §6.1.1 Pre-Processing Tools Sadness and the §6.1.2 python Sadness section.

#### 4.1.3. `ld`, resource files, and other vendor-specific link-time tools

Resource files and other "link time" or post-processing measures have one benefit over the previous method: they are fast to perform in terms of compilation time. A example can be seen in the §6.1.3 ld Sadness section.

#### 4.1.4. The `incbin` tool

There is a tool called [incbin] which is a 3rd party attempt at pulling files in at "assembly time". Its approach is incredibly similar to `ld`, with the caveat that files must be shipped with their binary. It unfortunately falls prey to the same problems of cross-platform woes when dealing with VC++, requiring additional pre-processing to work out in full.

### 4.2. Prior Art

There has been a lot of discussion over the years in many arenas, from Stack Overflow to mailing lists to meetings with the Committee itself. The latest advancements that had been brought to WG21's attention was p0373r0 - File String Literals. It proposed the syntax `F"my_file.txt"` and `bF"my_file.txt"`, with a few other amenities, to load files at compilation time. The following is an analysis of the previous proposal.

#### 4.2.1. Literal-Based, constexpr

A user could reasonably assign (or want to assign) the resulting array to a `constexpr` variable as its expected to be handled like most other string literals. This allowed some degree of compile-time reflection. It is entirely helpful that such file contents be assigned to constexpr: e.g., string literals of JSON being loaded at compile time to be parsed by Ben Deane and Jason Turner in their CppCon 2017 talk, constexpr All The Things.

#### 4.2.2. Literal-Based, Null Terminated (?)

It is unclear whether the resulting array of characters or bytes was to be null terminated. The usage and expression imply that it will be, due to its string-like appearance. However, is adding an additional null terminator fitting for desired usage? From the existing tools and practice (e.g., `xxd -i` or linking a data-dumped object file), the answer is no: but the syntax `bF"hello.txt"` makes the answer seem like a "yes". This is confusing: either the user should be given an explicit choice or the feature should be entirely opt-in.

#### 4.2.3. Encoding

Because the proposal used a string literal, several questions came up as to the actual encoding of the returned information. The author gave both `bF"my_file.txt"` and `F"my_file.txt"` to separate binary versus string-based arrays of returns. Not only did this conflate issues with expectations in the previous section, it also became a heavily contested discussion on both the mailing list group discussion of the original proposal and in the paper itself. This is likely one of the biggest pitfalls between separating "binary" data from "string" data: imbuing an object with string-like properties at translation time provide for all the same hairy questions around source/execution character set and the contents of a literal.

### 4.3. Design Goals

Because of the aforementioned reasons, it seems more prudent to take a "compiler intrinsic"/"magic function" approach. The function takes the form:

```
template <typename T = byte>
consteval span<const T> embed(
  string_view resource_identifier
);
```

`resource_identifier` is a `string_view` processed in an implementation-defined manner to find and pull resources into C++ at constexpr time. The most obvious source will be the file system, with the intention of having this evaluated as a core constant expression. We do not attempt to restrict the `string_view` to a specific subset: whatever the implementation accepts (typically expected to be a relative or absolute file path, but can be other identification scheme), the implementation should use.

### § 4.3.1. Implementation Defined

Calls such as `std::embed( "my_file.txt" );`, `std::embed( "data.dll" );`, and `std::embed<vertex>( "vertices.bin" );` are meant to be evaluated in a `constexpr` context (with "core constant expressions" only), where the behavior is implementation-defined. The function has unspecified behavior when evaluated in a non-constexpr context (with the expectation that the implementation will provide a failing diagnostic in these cases). This is similar to how include paths work, albeit `#include` interacts with the programmer through the preprocessor.

There is precedent for specifying library features that are implemented only through compile-time compiler intrinsics (`type_traits`, `source_location`, and similar utilities). Core -- for other proposals such as p0466r1 - Layout-compatibility and Pointer-interconvertibility Traits -- indicated their preference in using a `constexpr` magic function implemented by intrinsic in the standard library over some form of `template <auto X> thing { /* implementation specified */ value; };` construct. However, it is important to note that [p0466r1] proposes type traits, where as this has entirely different functionality, and so its reception and opinions may be different.

### § 4.3.2. Binary Only

Creating two separate forms or options for loading data that is meant to be a "string" always fuels controversy and debate about what the resulting contents should be. The problem is sidestepped entirely by demanding that the resource loaded by `std::embed` represents the bytes exactly as they come from the resource. This prevents encoding confusion, conversion issues, and other pitfalls related to trying to match the user's idea of "string" data or non-binary formats. Data is received exactly as it is from the resource as defined by the implementation, whether it is a supposed text file or otherwise. `std::embed( "my_text_file.txt" )` and `std::embed( "my_binary_file.bin" )` behave exactly the same concerning their treatment of the resource.

### § 4.3.3. Constexpr Compatibility

The entire implementation must be usable in a `constexpr` context. It is not just for the purposes of processing the data at compile time, but because it matches existing implementations that store strings and huge array literals into a variable via `#include`. These variables can be `constexpr`: to not have a constexpr implementation is to leave many of the programmers who utilize this behavior without a proper standardized tool.

### § 4.3.4. Statically Polymorphic

While returning `std::byte` is valuable, it is impossible to `reinterpret_cast` or `bit_cast` certain things at compile time. This makes it impossible in a `constexpr` context to retrieve the actual data from a resource without tremendous boilerplate and work that every developer will have to do.

## § 5. Changes to the Standard

Wording changes are relative to [n4778].

### § 5.1. Intent

The intent of the wording is to provide a function that:

— handles the provided resource identifying `string_view` in an implementation-defined manner;

— and, returns the specified constexpr `span` representing either the bytes of the resource or the bytes view as the type `T`.

The wording also explicitly disallows the usage of the function outside of a core constant expression by marking it `consteval`, meaning it is ill-formed if it is attempted to be used at not-constexpr time (`std::embed` calls should not show up as a function in the final executable or in generated code). The program may pin the data returned by `std::embed` through the span into the executable if it is used outside a core constant expression.

### § 5.2. Proposed Feature Test Macro

The proposed feature test macro is `__cpp_lib_embed`.

### § 5.3. Proposed Wording

Append to §16.3.1 General [**support.limits.general**]'s **Table 35** one additional entry:

> | Macro name | Value |
> |---|---|
> | __cpp_lib_embed | 201902L |

Append to §19.1 General [**utilities.general**]'s **Table 38** one additional entry:

> | | Subclause | Header(s) |
> |---|---|---|
> | 19.20 | Compile-time Resources | <embed> |

Add a new section §19.20 Compile-time Resources [**const.res**]:

**19.20 Compile-time Resources [const.res]**

**19.20.1** In general [**const.res.general**]

Compile-time resources allow the implementation to retrieve data into a program from implementation-defined sources.

**19.20.2** Header `embed` synopsis [**embed.syn**]

```
namespace std {
        template <typename T>
        consteval span<const T> embed( string_view resource_identifier ) noexcept;
}
```

**19.20.3** Function template `embed` [**embed.embed**]

```
namespace std {
        template <typename T = byte>
        consteval span<const T> embed( string_view resource_identifier ) noexcept;
}
```

[1] Constraints: `T` shall satisfy `std::is_trivial_v<T>`. [*Note*— This constraint ensures that types with non-trivial destructors do not need to be run for the compiler-provided unknown storage. — *end Note*].

[2] Returns: A contiguous sequence of `T` representing the resource provided by the implementation.

[3] Remarks: Accepts a `string_view` whose value is used to search a sequence of implementation-defined places for a resource identified uniquely by the resource identifier specified by the `string_view` argument. The entire contents are made available as a contiguous sequence of `T` in the returned `span`. If the implementation cannot find the resource specified after exhausting the sequence of implementation-defined search locations, the implementation shall error. [*Note*— Implementations should provide a mechanism similar to include paths to find the specified resource. — *end Note*]

# § 6. Appendix

## § 6.1. Sadness

Other techniques used include pre-processing data, link-time based tooling, and assembly-time runtime loading. They are detailed below, for a complete picture of today's sad landscape of options.

### § 6.1.1. Pre-Processing Tools Sadness

1. Run the tool over the data (`xxd -i xxd_data.bin > xxd_data.h`) to obtain the generated file (`xxd_data.h`):

```
unsigned char xxd_data_bin[] = {
  0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x2c, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0x64,
  0x0a
};
unsigned int xxd_data_bin_len = 13;
```

2. Compile `main.cpp`:

```cpp
#include <iostream>
#include <string_view>

// prefix as constexpr,
// even if it generates some warnings in g++/clang++
constexpr
#include "xxd_data.h"
;

template <typename T, std::size_t N>
constexpr std::size_t array_size(const T (&)[N]) {
    return N;
}

int main() {
    static_assert(xxd_data_bin[0] == 'H');
    static_assert(array_size(xxd_data_bin) == 13);

    std::string_view data_view(
        reinterpret_cast<const char*>(xxd_data_bin),
        array_size(xxd_data_bin));
    std::cout << data_view << std::endl; // Hello, World!
    return 0;
}
```

Others still use python or other small scripting languages as part of their build process, outputting data in the exact C++ format that they require.

There are problems with the `xxd -i` or similar tool-based approach. Lexing and Parsing data-as-source-code adds an enormous overhead to actually reading and making that data available.

Binary data as C(++) arrays provide the overhead of having to comma-delimit every single byte present, it also requires that the compiler verify every entry in that array is a valid literal or entry according to the C++ language.

This scales poorly with larger files, and build times suffer for any non-trivial binary file, especially when it scales into Megabytes in size (e.g., firmware and similar).

### § 6.1.2. `python` Sadness

Other companies are forced to create their own ad-hoc tools to embed data and files into their C++ code. MongoDB uses a custom python script, just to get their data into C++:

```
import os
import sys

def jsToHeader(target, source):
    outFile = target
    h = [
        '#include "mongo/base/string_data.h"',
        '#include "mongo/scripting/engine.h"',
        'namespace mongo {',
        'namespace JSFiles{',
    ]
    def lineToChars(s):
        return ','.join(str(ord(c)) for c in (s.rstrip() + '\n')) + ','
    for s in source:
        filename = str(s)
        objname = os.path.split(filename)[1].split('.')[0]
        stringname = '_jscode_raw_' + objname

        h.append('constexpr char ' + stringname + "[] = {")

        with open(filename, 'r') as f:
            for line in f:
                h.append(lineToChars(line))

        h.append("0};")
        # symbols aren't exported w/o this
        h.append('extern const JSFile %s;' % objname)
        h.append('const JSFile %s = { "%s", StringData(%s, sizeof(%s) - 1) };' %
                 (objname, filename.replace('\\', '/'), stringname, stringname))

    h.append("} // namespace JSFiles")
    h.append("} // namespace mongo")
    h.append("")

    text = '\n'.join(h)

    with open(outFile, 'wb') as out:
        try:
            out.write(text)
        finally:
            out.close()


if __name__ == "__main__":
    if len(sys.argv) < 3:
        print "Must specify [target] [source] "
        sys.exit(1)
    jsToHeader(sys.argv[1], sys.argv[2:])
```

MongoDB were brave enough to share their code with me and make public the things they have to do: other companies have shared many similar concerns, but do not have the same bravery. We thank MongoDB for sharing.

### § 6.1.3. `ld` Sadness

A full, compilable example (except on Visual C++):

0. Have a file ld_data.bin with the contents `Hello, World!`.
1. Run `ld -r binary -o ld_data.o ld_data.bin`.
2. Compile the following `main.cpp` with `c++ -std=c++17 ld_data.o main.cpp`:

```
#include <iostream>
#include <string_view>

#ifdef __APPLE__
#include <mach-o/getsect.h>

#define DECLARE_LD(NAME) extern const unsigned char _section$__DATA__##NAME[];
#define LD_NAME(NAME) _section$__DATA__##NAME
#define LD_SIZE(NAME) (getsectbyname("__DATA", "__" #NAME)->size)

#elif (defined __MINGW32__) /* mingw */

#define DECLARE_LD(NAME)                                    \
  extern const unsigned char binary_##NAME##_start[]; \
  extern const unsigned char binary_##NAME##_end[];
#define LD_NAME(NAME) binary_##NAME##_start
#define LD_SIZE(NAME) ((binary_##NAME##_end) - (binary_##NAME##_start))

#else /* gnu/linux ld */

#define DECLARE_LD(NAME)                                     \
  extern const unsigned char _binary_##NAME##_start[]; \
  extern const unsigned char _binary_##NAME##_end[];
#define LD_NAME(NAME) _binary_##NAME##_start
#define LD_SIZE(NAME) ((_binary_##NAME##_end) - (_binary_##NAME##_start))
#endif

DECLARE_LD(ld_data_bin);

int main() {
  // impossible
  //static_assert(xxd_data_bin[0] == 'H');
  std::string_view data_view(
    reinterpret_cast<const char*>(LD_NAME(ld_data_bin)),
    LD_SIZE(ld_data_bin)
  );
  std::cout << data_view << std::endl; // Hello, World!
  return 0;
}
```

This scales a little bit better in terms of raw compilation time but is shockingly OS, vendor and platform specific in ways that novice developers would not be able to handle fully. The macros are required to erase differences, lest subtle differences in name will destroy one's ability to use these macros effectively. We ommitted the code for handling VC++ resource files because it is excessively verbose than what is present here.

N.B.: Because these declarations are `extern`, the values in the array cannot be accessed at compilation/translation-time.

## § 7. Acknowledgements

A big thank you to Andrew Tomazos for replying to the author's e-mails about the prior art. Thank you to Arthur O'Dwyer for providing the author with incredible insight into the Committee's previous process for how they interpreted the Prior Art.

A special thank you to Agustín Bergé for encouraging the author to talk to the creator of the Prior Art and getting started on this. Thank you to Tom Honermann for direction and insight on how to write a paper and apply for a proposal.

Thank you to Arvid Gerstmann for helping the author understand and use the link-time tools.

Thank you to Tony Van Eerd for valuable advice in improving the main text of this paper.

Thank you to Lilly (Cpplang Slack, @lillypad) for the valuable bikeshed and hole-poking in original designs, alongside Ben Craig who very thoroughly explained his woes when trying to embed large firmware images into a C++ program for deployment into production.

For all this hard work, it is the author's hope to carry this into C++. It would be the author's distinct honor to make development cycles easier and better with the programming language we work in and love. ♥

## § References

### § Informative References

**[CONSTEXPR-ALL-THE-THINGS]**
Ben Deane; Jason Turner. constexpr All The Things: CppCon 2017. September 25th, 2017. URL: https://www.youtube.com/watch?v=PJwd4JLYJJY

**[INCBIN]**
Dale Weiler (graphitemaster). incbin: load files at 'assembly' time. URL: https://github.com/graphitemaster/incbin

**[N4778]**
ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4778 - Working Draft, Standard for Programming Language C++. November 26th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf

**[NONIUS-VISUAL-C-ERROR]**
R. Martinho Fernandes. nonius generated HTML Reporter. September 1st, 2016. URL: https://github.com/libnonius/nonius/blob/devel/include/nonius/reporters/html_reporter.h%2B%2B#L42

**[P0373R0]**
Andrew Tomazos. Proposal of File Literals. 21 May 2016. URL: https://wg21.link/p0373r0

**[P0466R1]**
Lisa Lippincott. Layout-compatibility and Pointer-interconvertibility Traits. 12 February 2018. URL: https://wg21.link/p0466r1

**[P1130]**
JeanHeyd Meneide. Module Resource Requirement Propagation. November 26th, 2018. URL: https://thephd.github.io/vendor/future_cxx/papers/d1130.html